

Practical Deep Learning

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

- ▶ PyTorch CNN Implementation
- ▶ Data Handling
- ▶ Data Augmentation
- ▶ Transfer Learning
- ▶ Ensembling
- ▶ Dropout
- ▶ Batch Normalization
- ▶ Full Training Workflow
- ▶ Evaluation Metrics

- ▶ Build and train a convolutional network in PyTorch
- ▶ Implement efficient data handling techniques using PyTorch's Dataset and DataLoader classes
- ▶ Apply appropriate data augmentation strategies based on error analysis
- ▶ Utilize transfer learning effectively for different scenarios and dataset sizes
- ▶ Understand ensemble methods to improve model performance
- ▶ Apply regularization techniques like Dropout and Batch Normalization
- ▶ Design a complete deep learning training workflow from initial setup to inference
- ▶ Choose and interpret evaluation metrics that suit the task and the class balance

- ▶ Practical Implementation of Deep Learning algorithms is just as much an art as it is a science.
- ▶ The main takeaway is not to start from scratch but rather to build on top of previous knowledge.
- ▶ Today, we will look at some important tools used in the practical implementation of Deep Learning algorithms.

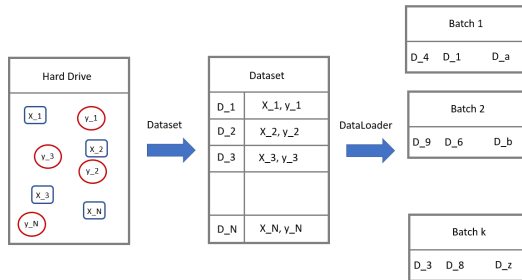
CNN: Implementing CNNs in PyTorch

```
import torch
import torch.nn as nn
import torch.nn.functional as F
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(64 * 8 * 8, 128) # assumes 32x32 input (e.g. CIFAR-10)
        self.fc2 = nn.Linear(128, 10) # assuming 10 classes
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 64 * 8 * 8)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

```
import torch.optim as optim
model = SimpleCNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
# Training loop
for epoch in range(10):
    for images, labels in dataloader:
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

- ▶ As we have previously established, Deep Learning has been made possible by large amounts of data and computational resources.
- ▶ An important aspect to keep in mind is the data handling:
 - How do we handle large amounts of data?
 - How do we read different components of data (from possibly different parts of our hard drive) and provide it to our training algorithms?
 - How do we feed this data to SGD algorithms in a streamlined manner?
- ▶ PyTorch provides Dataset and DataLoaders to handle data in an efficient manner.
- ▶ We will subclass Dataset to describe our own data, then hand it to a DataLoader for batching and shuffling.

- ▶ Datasets in PyTorch: The `torch.utils.data.Dataset` class is an abstract base class used to define and manage datasets for training and evaluation.
- ▶ Custom Dataset Creation: You can create a custom dataset by subclassing `Dataset` and implementing
 - `__len__()` (returns dataset size) and
 - `__getitem__()` (fetches a single data sample).
- ▶ DataLoaders for Batching: `torch.utils.data.DataLoader` is used to load data in mini-batches, shuffle data, and utilize multiprocessing for efficiency.
- ▶ Built-in Datasets: PyTorch provides datasets in `torchvision.datasets`, making it easy to load common datasets like MNIST and CIFAR-10.



```
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
class ImageDataset(Dataset):
    def __init__(self, paths, labels, transform=None):
        self.paths = paths
        self.labels = labels
        self.transform = transform
    def __len__(self): # how many samples the dataset holds
        return len(self.paths)
    def __getitem__(self, idx): # fetch one sample
        image = Image.open(self.paths[idx]).convert("RGB")
        if self.transform:
            image = self.transform(image)
        return image, self.labels[idx]
```

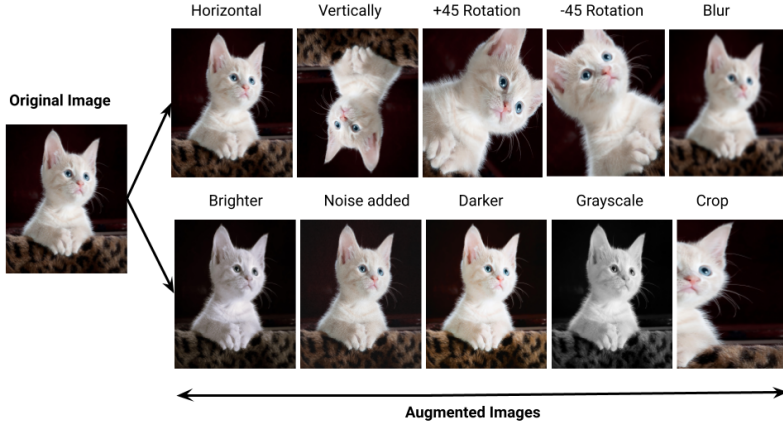
```
to_tensor = transforms.Compose([transforms.ToTensor()])
train_set = ImageDataset(paths, labels, transform=to_tensor)
train_loader = DataLoader(train_set, batch_size=64, shuffle=True,
                          num_workers=4)
```

- ▶ Only `__len__()` and `__getitem__()` are ours to write; `__getitem__()` reads one sample from disk, so nothing beyond the current batch is held in memory.
- ▶ The `DataLoader` does the rest: it groups samples into batches, reshuffles them each epoch, and reads ahead in `num_workers` background processes so the GPU is not left waiting.

- ▶ Data is the fundamental building block of any machine learning algorithm
- ▶ In several applications we don't have access to unlimited data
- ▶ So we use Data Augmentation techniques to improve the performance of our models
- ▶ Note: It is better to spend time on data rather than fine-scale architecture search in deep learning

► Create virtual training samples

- Horizontal flip
- Random crop
- Color casting
- Geometric distortion
- Translation
- Rotation



⁰Figure: Pranjal Ostwal, "Data Augmentation for Computer Vision" (Medium)



⁰Figure: albumentations transform demo (<https://github.com/albumentations-team/albumentations>; now archived, successor: AlbumentationsX). Library: Buslaev et al., “Albumentations: Fast and Flexible Image Augmentations”, *Information* 11(2):125, 2020.

- ▶ But, there are many types of augmentations. How do we choose the right ones for our task?

- ▶ But, there are many types of augmentations. How do we choose the right ones for our task?
- ▶ **Answer: Error Analysis** – Identify model weaknesses and apply augmentations that address those issues.

► Steps:

1. Train a baseline model.
2. Make predictions on validation data.
3. Inspect the worst predictions to identify model weaknesses.
4. Apply relevant augmentations to address these issues.

► Examples:

- **Failure with small objects** → Use *Scale Augmentation*.
- **Failure with different colors/environments** → Use *Color Augmentations*.
- **Failure with rotated images** → Use *Rotation Augmentations*.
- **Failure with blurry images** → Use *Blur Augmentations* (e.g., GaussianBlur, MotionBlur).
- ... etc.

- ▶ **Note:** Data augmentation is applied only to the training data.
- ▶ Applying it to validation/test data **directly** would lead to **incorrect evaluation and misleading performance metrics**.

- However, there is a special inference technique called **Test-Time Augmentation (TTA)** where multiple augmented versions of the same image are passed through the model separately, and the predictions are averaged to improve accuracy.

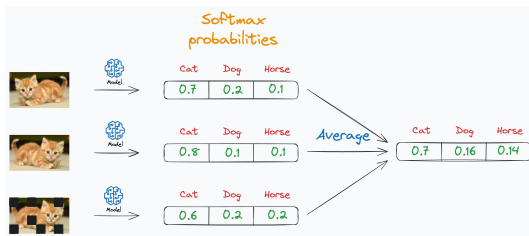


Figure 1: Test-time Augmentation

⁰Figure: Avi Chawla & Baniyas Baabe, "Train and Test-time Data Augmentation", Daily Dose of Data Science, 2024.

- ▶ TTA is an advanced technique.
- ▶ Applying it requires **custom scripts** to generate augmented versions of test images, pass them through the model, and aggregate predictions.

► What is Fine-Tuning?

- A strategy in **transfer learning** where a pre-trained model is adapted to a new task.
- Instead of training from scratch, we start from a model already trained on a related task.

► Why use Fine-Tuning?

- Saves time and computational resources.
- Improves performance, especially with limited data.

- ▶ New dataset is small + similar distribution to original dataset:
 - Freeze (or partially freeze) feature extraction layers and fine-tune the classifier.
- ▶ New dataset is small + different distribution to original dataset:
 - Use the pretrained network as a generic feature extractor and train a light classifier on top (e.g., SVM).
 - In modern practice, you might also freeze earlier layers and selectively fine-tune later layers.
- ▶ New dataset is large, regardless of the original data distribution:
 - Fine-tune the entire network (both feature extractor and classifier).

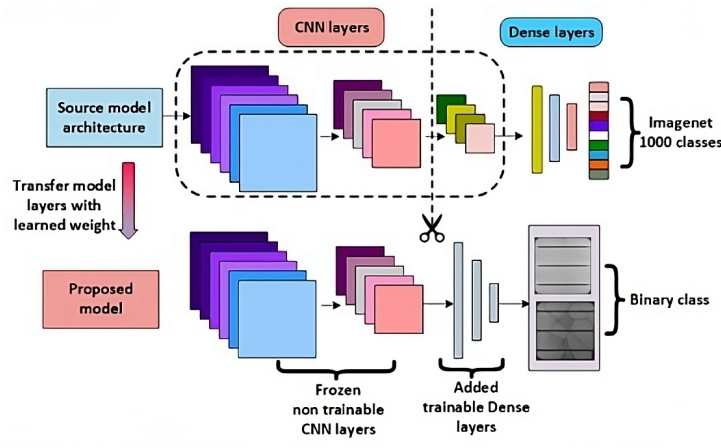


Figure 2: Fine-tuning a pretrained ImageNet model for a new task

⁰Figure: Rahman, Tabassum, Haque, Nishat, Faisal & Hossain, "CNN-based Deep Learning Approach for Micro-crack Detection of Solar Panels", STI 2021, Fig. 3.

- ▶ No single model is perfect. Different models make different types of errors.
- ▶ Let's visualize the predictions of two different models:

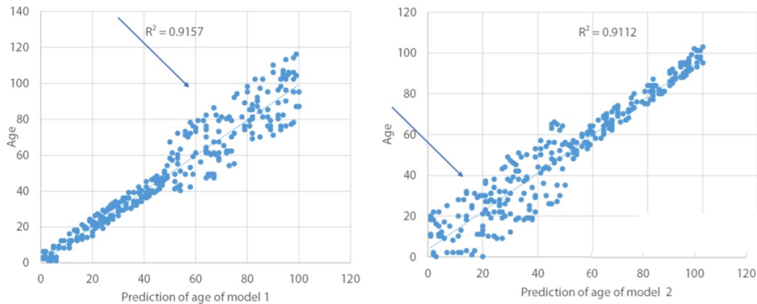


Figure 3: Predictions of Model 1 and Model 2

⁰Figure: Coursera / HSE, “How to Win a Data Science Competition: Learn from Top Kagglers”, week 4 ensembling lecture.

- Combining predictions of diverse models can reduce errors and improve accuracy. This technique is called **Ensembling**.

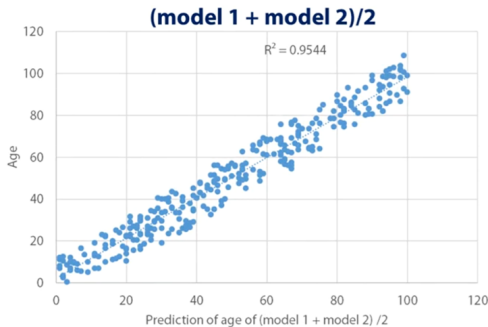


Figure 4: Averaged Predictions: Better Correlation

⁰Figure: Coursera / HSE, “How to Win a Data Science Competition: Learn from Top Kagglers”, week 4 ensembling lecture.

- Diverse models are key to effective ensembling. Here are two strategies:
 - **Bagging (Bootstrap Aggregating):** Train multiple models on different subsets of data (e.g., Random Forest model).

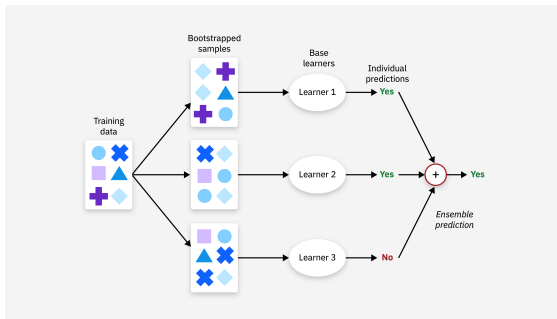


Figure 5: Bagging example

⁰Figure: IBM, <https://www.ibm.com/think/topics/bagging>

► Boosting:

- Train models sequentially, where each model focuses on the errors of the previous one (e.g., AdaBoost, Gradient Boosting).

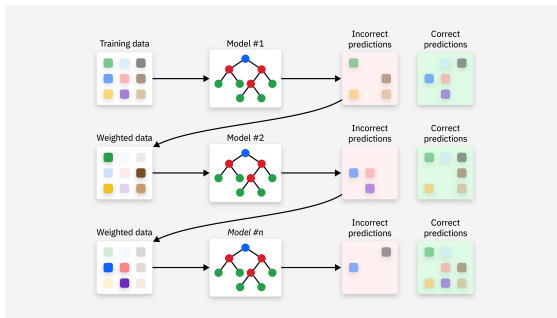


Figure 6: Boosting Example

⁰Figure: IBM, <https://www.ibm.com/think/topics/boosting>

- ▶ These strategies introduce diversity in models, but how can we combine their predictions?

- ▶ We can combine classifiers' predictions in two ways:
 - **Hard Voting:**
 - ▶ Each model votes for a class.
 - ▶ The class with the majority of votes is selected.
 - **Soft Voting:**
 - ▶ Average the predicted probabilities from each model.
 - ▶ The class with the highest average probability is selected.
- ▶ For regressors: take the average of predictions from all models.

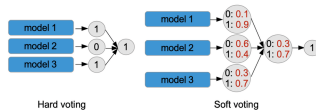


Figure 7: Illustration of Hard and Soft Voting for Classifiers

⁰Figure: Bulaty Tautil et al., *Rev. Observatorio de la Economía Latinoamericana* 22(5), 2024, Fig. 2.

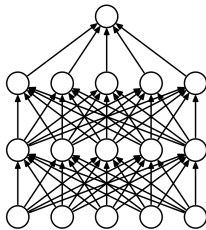
- ▶ Teamwork is the best policy.
- ▶ We can train multiple networks for the same task then ensemble to get better results.

- ▶ Let's assume that we have a test dataset with N elements and an ensemble of M models.
- ▶ Also assume that each model in the ensemble errs independently on an image with the same probability, denoted by e .
- ▶ For example, assume $M = 3$ and $e = 0.01$.
- ▶ Then the probability of error of the label for the max-voting ensemble will be

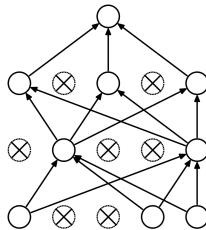
$$p_{\text{ens}} = 1 - (1 - e)^3 - \binom{3}{2}(1 - e)^2 e$$

- ▶ For the above example $p_{\text{ens}} = 0.0003$, which is significantly lower than a single model.

- ▶ **Dropout** is a regularization technique used to prevent overfitting in neural networks.
- ▶ During training, it randomly drops (sets to 0) a fraction of neurons in each layer based on a specified probability for every forward pass.

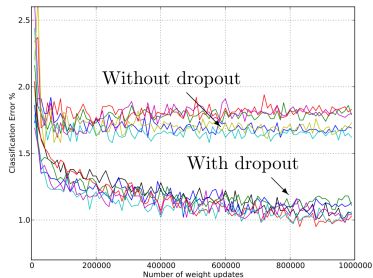


(a) Standard Neural Net



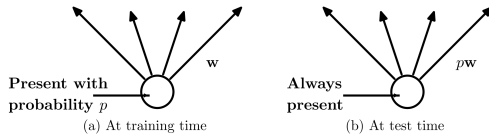
(b) After applying dropout.

- ▶ Why is dropping neurons randomly useful?
 1. By dropping a subset of neurons during each forward pass, the network learns **not to rely too heavily on specific connections**, encouraging the network to use more connections.
 2. Dropout acts as an **efficient ensemble** of multiple smaller networks, each trained on a random subset of neurons.
 3. More Connections + Diverse Ensemble = **less overfitting**.



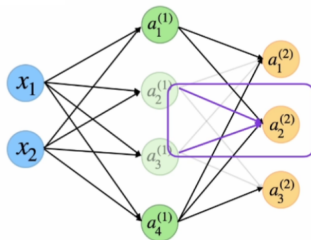
Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
SVM on Fisher Vectors of Dense SIFT and Color Statistics	-	-	27.3
Avg of classifiers over FVs of SIFT, LBP, GIST and CSIFT	-	-	26.2
Conv Net + dropout (Krizhevsky et al., 2012)	40.7	18.2	-
Avg of 5 Conv Nets + dropout (Krizhevsky et al., 2012)	38.1	16.4	16.4

Table 6: Results on the ILSVRC-2012 validation/test set.



- ▶ We drop neurons during training, but what should we do during inference?
- ▶ **During inference:** We use all neurons.

But...



Assume we have
present probability
 $p = 0.5$

During testing,
when nodes are
present again,
activations are
 $2 \times$ as large!

- ▶ **Problem:** Having inconsistent values during inference compared to training can cause the network to produce worse results.
- ▶ **Solution:** Scale the output during inference by the **keep probability** p (the chance a neuron stays active) to keep the same expected activation as in training.
- ▶ **Example:**
 - **During training (keep probability $p = 0.1$):**
 - ▶ Activation value = 2
 - ▶ Active only 10% of the time
 - ▶ $\Rightarrow$ Effective contribution = $2 \times 0.1 = 0.2$
 - **During inference:**
 - ▶ Always active ($\Rightarrow$ Activation value = 2)
 - ▶ Scale by $p = 0.1 \Rightarrow 2 \times 0.1 = 0.2$
 - **Result:** Consistent activation scale between training and inference.

- In PyTorch, dropout behavior is controlled by the model's mode:

```
# Training Mode: Enables dropout
model.train()
output = model(input)
```

```
# Evaluation Mode: Disables dropout (no scaling needed;
# PyTorch already scales by 1/(1-p) during training)
model.eval()
output = model(input)
```

- **Note:** `nn.Dropout(p)` takes the *drop* probability, so a keep probability of 0.1 is written `nn.Dropout(0.9)`.

- ▶ **How much to drop:** a keep probability of $p = 0.5$ on the hidden layers is close to optimal across most networks and tasks, and the test error stays flat for $0.4 \leq p \leq 0.8$. Input layers want a higher keep probability, around 0.8.
- ▶ **It costs training time:** a dropout network typically takes 2 to 3 times longer to train than the same architecture without it, because each update is computed on a different thinned network and so is much noisier.
- ▶ **It does not always help:** with very little training data the network can memorise the training set even through the dropout noise. On MNIST subsets of 100 and 500 images, dropout gave no improvement.

⁰Srivastava JMLR 2014, Sections 7.3, 7.4 and 11

- ▶ **Key Insight:** Networks train better when inputs are normalized
- ▶ So why not normalize intermediate layers too?
- ▶ **Problem:** forcing every layer to output zero-mean, unit-variance values might be too restrictive (not always optimal).
- ▶ **Solution:** Let the network learn if it wants normalization!

For each feature in a layer:

1. First normalize: $\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$
 - μ : mean across batch dimension
 - σ^2 : variance across batch dimension
 - Computed separately for each feature/channel
2. Then give the network control: $y = \gamma \hat{x} + \beta$
 - γ (scale): Can amplify or reduce the normalized values
 - β (shift): Can move the values away from zero
 - These are learned during training like normal weights!

- ▶ After normalization, outputs are always zero-mean, unit-variance
- ▶ But this might not be optimal for every layer!
- ▶ γ and β let each layer learn:
 - γ : “How much variance do I want?”
 - β : “What should my mean activation be?”
- ▶ Two extremes the network can learn:
 - “Keep normalization”: $\gamma \approx 1$, $\beta \approx 0$
 - “Undo normalization”: γ and β restore original scale and shift
- ▶ Network learns what’s best for each layer!

Mathematically, for batch size N , at each feature/channel j :

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j} \quad (\text{batch mean})$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2 \quad (\text{batch variance})$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \quad (\text{normalize})$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j \quad (\text{learnable transform})$$

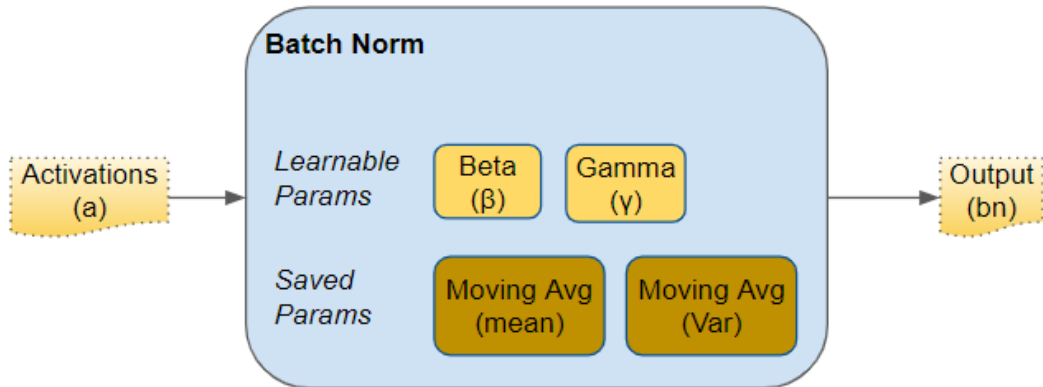
► During Training:

- Use statistics from current batch
- Keep running average of mean and variance (to use later in inference):

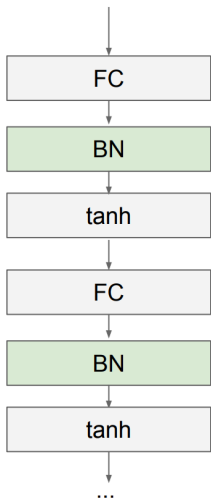
$$\mu_{running} = 0.9 \times \mu_{running} + 0.1 \times \mu_{batch}$$

► During Inference (We can't depend on mini-batches):

- Use stored running averages ($\mu_{running}, \sigma_{running}^2$)
- These are fixed values from training
- No batch statistics needed!



⁰Figure: Ketan Doshi, "Batch Norm Explained Visually" ([towardsdatascience.com](https://towardsdatascience.com/batch-normalization-explained-visually))



Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

⁰Figure: CS231n, Stanford University

- In PyTorch, batch normalization behavior is controlled by the model's mode:

```
# Training Mode: Use batch statistics  
model.train()  
output = model(input)
```

```
# Evaluation Mode: Use running statistics  
model.eval()  
output = model(input)
```

► Advantages:

- Makes deep networks much easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!

⁰Slide based on [Stanford CS231n](#) by Fei-Fei Li, Yunzhu Li & Ruohan Gao

► Advantages:

- Makes deep networks much easier to train!
- Improves gradient flow
- Allows higher learning rates, faster convergence
- Networks become more robust to initialization
- Acts as regularization during training
- Zero overhead at test-time: can be fused with conv!

► Disadvantages:

- Behaves differently during training and testing: this is a very common source of bugs!

⁰Slide based on [Stanford CS231n](#) by Fei-Fei Li, Yunzhu Li & Ruohan Gao

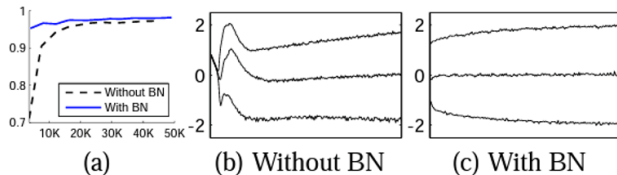


Figure 1: (a) *The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy.* (b, c) *The evolution of input distributions to a typical sigmoid, over the course of training, shown as $\{15, 50, 85\}$ th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.*

- ▶ Both reduce overfitting, so it is tempting to switch on both. They interfere.
- ▶ Dropout changes the variance of a unit between training and test time. Batch normalization stores the variance it measured during training and reuses that stored value at test time, so the two disagree about the same quantity.
- ▶ The disagreement compounds from layer to layer and shows up as a drop in test accuracy. It has been measured across a range of modern convolutional networks.
- ▶ **Practical rule:** put dropout after all batch normalization layers, usually in the fully connected head, rather than between a convolution and its normalization. Many modern architectures get their best results from batch normalization with no dropout at all.

⁰Li, Chen, Hu & Yang, “Understanding the Disharmony between Dropout and Batch Normalization by Variance Shift”, CVPR 2019

► Step 1: Initial Setup

- Start with a pretrained model and just finetune when possible; it saves time and improves results.
- Define an initial architecture without regularization (e.g., dropout) or augmentations.
- Set up validation strategy and choose an appropriate evaluation loss and metric.
- Train the model to get a **baseline score**.

► Step 2: Improvement Process

- Overfitting is common at the start; use regularization techniques like:
 - Dropout, batch normalization,...
- Perform error analysis to identify weaknesses and choose appropriate augmentations or preprocessing.
- Tune hyperparameters: layers, epochs, learning rate, batch size, etc.
- Optionally, use ensembling to boost scores (requires more resources).

► **Key Tip:** Track scores and improvements at every step to measure progress effectively.

► Step 3: Finalization

- Save the optimized model for deployment.
- Use the model for inference in real-world applications.

Accuracy:

- ▶ The ratio of correctly predicted instances to the total instances.
- ▶ It is a common metric for classification tasks.
- ▶ Formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- ▶ Where:
 - TP: True Positives
 - TN: True Negatives
 - FP: False Positives
 - FN: False Negatives
- ▶ Limitations:
 - It can be misleading in imbalanced datasets.
 - It does not provide information about the types of errors made.

Precision:

- ▶ The ratio of correctly predicted positive instances to the total predicted positive instances.
- ▶ Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- ▶ It indicates how many of the predicted positive instances were actually positive.
- ▶ High precision means fewer false positives.
- ▶ Limitations:
 - It does not consider false negatives.
 - It can be misleading in imbalanced datasets.

Recall:

- ▶ The ratio of correctly predicted positive instances to the total actual positive instances.
- ▶ Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

- ▶ It indicates how many of the actual positive instances were predicted correctly.
- ▶ High recall means fewer false negatives.
- ▶ Limitations:
 - It does not consider false positives.
 - It can be misleading in imbalanced datasets.

F1 Score:

- ▶ The harmonic mean of precision and recall.
- ▶ Formula:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- ▶ It provides a balance between precision and recall.
- ▶ Useful for imbalanced datasets where one class is more important than the other.
- ▶ Limitations:
 - It can be misleading if the precision and recall are very different.
 - It does not provide information about the types of errors made.

Confusion Matrix:

- ▶ A table that summarizes the performance of a classification model.
- ▶ It shows the true positive, true negative, false positive, and false negative counts.
- ▶ It provides a detailed breakdown of the model's performance.
- ▶ Useful for understanding the types of errors made by the model.
- ▶ Can be used to calculate other metrics like accuracy, precision, recall, and F1 score.
- ▶ Example:

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

- ▶ Where:
 - TP: True Positives
 - TN: True Negatives
 - FP: False Positives
 - FN: False Negatives

mAP:

- ▶ mAP is calculated by averaging the average precision (AP) across all classes.
- ▶ AP is calculated by plotting the precision-recall curve and computing the area under the curve.
- ▶ mAP is particularly useful for evaluating models on datasets with multiple classes.
- ▶ It provides a comprehensive measure of the model's performance across all classes.
- ▶ Example:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i$$

- ▶ Where:
 - N is the number of classes.
 - AP_i is the average precision for class i .
- ▶ mAP can be calculated at different IoU thresholds (Intersection over Union: the overlap between the predicted and true boxes; e.g., 0.5, 0.75) to evaluate the model's performance at different levels of overlap.

Listing 1: Code snippet (PyTorch)

```
from torcheval.metrics.functional import multiclass_precision,  
    multiclass_recall, multiclass_f1_score  
  
p = multiclass_precision(preds, labels)  
r = multiclass_recall(preds, labels)  
f1= multiclass_f1_score(preds, labels)
```

These slides have been adapted from

- ▶ Fei-Fei Li, Yunzhu Li & Ruohan Gao, Stanford CS231n: [Deep Learning for Computer Vision](#)
- ▶ Sebastian Raschka, Lightning AI: [Deep Learning Fundamentals](#)
- ▶ PyTorch documentation: [torch.utils.data](#)

Primary sources for the methods covered

- ▶ Srivastava, Hinton, Krizhevsky, Sutskever & Salakhutdinov, [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#), JMLR 15, 2014
- ▶ Ioffe & Szegedy, [Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift](#), ICML 2015
- ▶ Li, Chen, Hu & Yang, [Understanding the Disharmony between Dropout and Batch Normalization by Variance Shift](#), CVPR 2019
- ▶ Buslaev, Iglovikov, Khvedchenya, Parinov, Druzhinin & Kalinin, [Albumentations: Fast and Flexible Image Augmentations](#), Information 11(2):125, 2020